

This command gets the most recent version of DuckDB and installs it so that you can use it in your Python environment. You can use DuckDB in your Python scripts or notebooks as soon as it is installed. The main interface is meant to be easy to use and similar to working with SQL. You can bring DuckDB into your program and use it to run SQL queries directly on data, whether it's stored in memory or in a database file that doesn't change.

You can connect like this to make an in-memory database, which is great for testing or doing temporary analysis:

```
import duckdb
conn = duckdb.connect('memory')
```

This makes a temporary database that only exists in RAM and is deleted when the connection closes. You can run SQL commands, and read and write data without having to worry about managing files. To create a file-based database that will last, give it a name:

```
conn = duckdb.connect('my_database.duckdb')
```

This makes or opens a file on disk called 'my_database.duckdb'. This file keeps track of all the changes you make to the data and schema, so you can keep your database up to date between sessions. The connection object lets you run SQL commands, which is the most basic use -- for instance, making a table, putting data in it, and asking questions:

```
conn.execute("CREATE TABLE users (id INTEGER, name VARCHAR)")
conn.execute("INSERT INTO users VALUES (1, 'Alice'), (2, 'Bob')")
result = conn.execute("SELECT * FROM users").fetchall()
print(result)
```

You can also work directly with pandas DataFrames, which makes it easy to analyse data:

```
import pandas as pd
df = pd.DataFrame({'id': [1, 2], 'name': ['Alice', 'Bob']})
conn.register('my_table', df)
result_df = conn.execute("SELECT * FROM my_table").fetchdf()
```

DuckDB makes it very easy to load and query data. It works with many different data formats and works well with pandas. To load data, DuckDB has simple SQL commands and functions that make it easy to work with CSV, Parquet, JSON, and DataFrames.

Reading data files

You can use the `READ_CSV`, `READ_PARQUET`, and `READ_JSON` functions to read common data formats directly into DuckDB. For instance, to load a CSV file:

```
CREATE TABLE my_csv AS SELECT * FROM read_csv_auto('data.csv');
```

The `read_csv_auto` function automatically finds the schema, which makes things easier. The same goes for Parquet files:

```
CREATE TABLE my_parquet AS SELECT * FROM read_parquet('data.parquet');
```

And for files in JSON format:

```
CREATE TABLE my_json AS SELECT * FROM read_json('data.json');
```

These commands make tables right from external files, which makes it easy to explore data quickly without having to import it into a separate database engine.

Querying pandas DataFrames

DuckDB lets you query pandas DataFrames directly, which makes it easy to use Python data in SQL workflows:

```
import pandas as pd
import duckdb
df = pd.read_csv('data.csv')
con = duckdb.connect()
con.register('df_view', df)
result = con.execute("SELECT * FROM df_view WHERE column_
value > 100").fetchdf()
```

This integration lets you run SQL queries on pandas DataFrames that are stored in memory. It combines the power of SQL with the flexibility of Python.

More advanced SQL operations

DuckDB has filtering, aggregation, joins, and window functions that make it easier to analyse complex data. For example:

Filtering:

```
SELECT * FROM my_table WHERE sales > 1000;
```

Groupings:

```
SELECT category, SUM(sales) FROM my_table GROUP BY category;
```

Joins:

```
SELECT a.*, b.* FROM table_a a JOIN table_b b WHERE a.id = b.id;
```

Functions for windows: